

DAY 5

MONGODB

MONGOOSE

PRODUCTION

Node.js: Everything You Need to Know

Day 5: MongoDB, Debugging & Production Practices

Upgrade the Real-Time Task Board from file storage to MongoDB, then learn to debug, monitor, and harden a Node.js application for production.

By **Wesam Abousaid**

Table of Contents

Objectives — What you'll be able to do by end of day

The Big Picture — Why databases replace files

Block 1 — MongoDB & Mongoose fundamentals

Block 2 — Swapping the storage layer (zero service-layer changes)

Block 3 — Debugging Node.js in VS Code

Block 4 — Monitoring, health, and performance

Block 5 — Production hardening

Hands-On Labs — Build the production-ready task board

By the End of Day 5, You Will Be Able To...



File-based Storage vs. Database

Explain why file-based storage breaks down and when a database is the right tool



Connect Node.js to MongoDB

Connect a Node.js app to MongoDB using Mongoose



Define Mongoose Schemas

Define Mongoose schemas and models with validation



Swap Storage Layer

Swap the file storage layer for MongoDB with zero changes to the service or socket layers



Debug Node.js Process

Debug a running Node.js process using VS Code breakpoints and the `--inspect` flag



Monitor Application Health

Monitor application health: memory, uptime, event loop lag



Apply Production Hardening

Apply production hardening: graceful shutdown, environment variables, error boundaries

The Big Picture: Why MongoDB Replaces Files

Day 5 closes the loop. The architecture you built across Days 1–4 was deliberately layered so that today's change is as small as possible.

Day 1 CLI → tasks.json (file)
Day 2 HTTP Server → tasks.json (file)
Day 3 Express + Modules → tasks.json (file, modular storage layer)
Day 4 Socket.IO → tasks.json (file, unchanged storage layer)
Day 5 MongoDB → Replace fileStorage.js only. Everything **else** stays.

"A good architecture makes the hard things easy and makes the wrong things obvious."

Why files break down

Problem	File Storage	MongoDB
Concurrent writes	Race conditions, data loss	Atomic operations
Querying	Load all, filter in memory	Index-based, fast
Multiple servers	Each server has its own file	Shared, centralised
Data size	Memory-limited	Disk-based, scalable
Schema validation	Manual, easy to skip	Enforced by Mongoose

MongoDB & Mongoose

What is MongoDB?

MongoDB stores data as documents — JSON-like objects — grouped into collections. There is no fixed schema enforced by the database itself, but Mongoose adds a schema layer in Node.js.

Relational DB	MongoDB equivalent
Database	Database
Table	Collection
Row	Document
Column	Field

A task document in MongoDB

```
{
  "_id": "665f1a2b3c4d5e6f7a8b9c0d",
  "title": "Buy milk",
  "done": false,
  "createdAt": "2025-06-04T09:00:00.000Z"
}
```

The `_id` field is a MongoDB ObjectId — a 12-byte unique identifier generated automatically. You do not need `Date.now()` for IDs anymore.



What is Mongoose?

Mongoose is an ODM (Object Document Mapper) for MongoDB. It sits between your application and the database and adds:

- Schemas — define the shape and types of documents
- Validation — reject invalid data before it reaches the DB
- Models — provide a typed interface for CRUD operations
- Middleware — pre/post hooks for save, delete, etc.

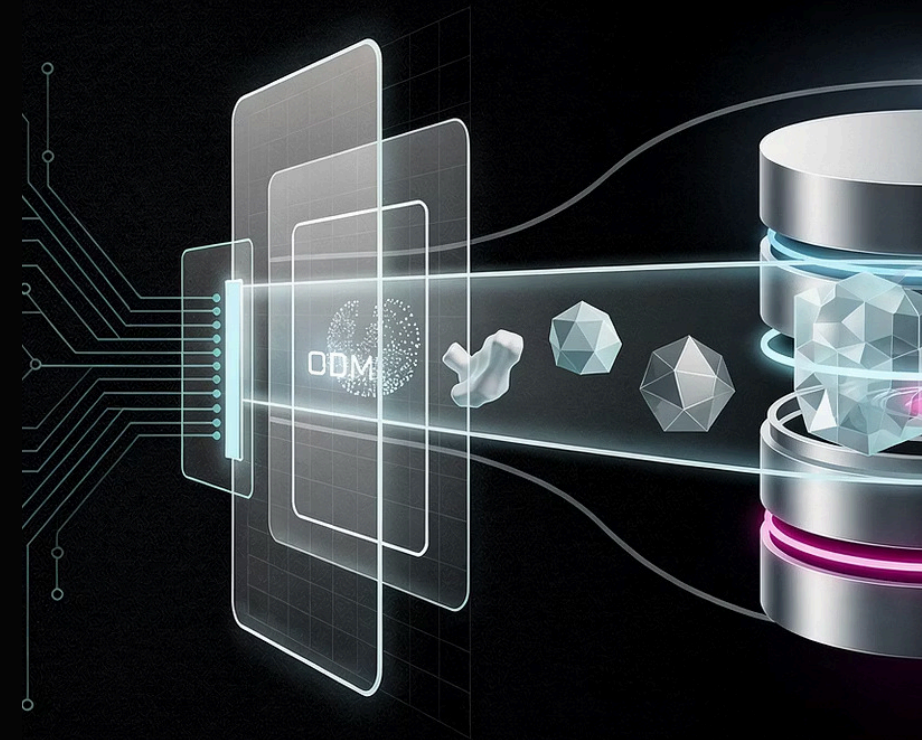
Raw MongoDB driver vs Mongoose

```
// Raw MongoDB driver
const task = await
db.collection('tasks').insertOne
({ title: 'Buy milk', done:
false });
```

```
// Mongoose
const task = await
Task.create({ title: 'Buy milk'
}); // schema fills in 'done:
false'
```

📌 Mongoose wins on validation, readability, and maintainability. Use the raw driver only when you need fine-grained performance tuning.

PACER



Connecting to MongoDB

```
// storage/db.js
const mongoose = require('mongoose');

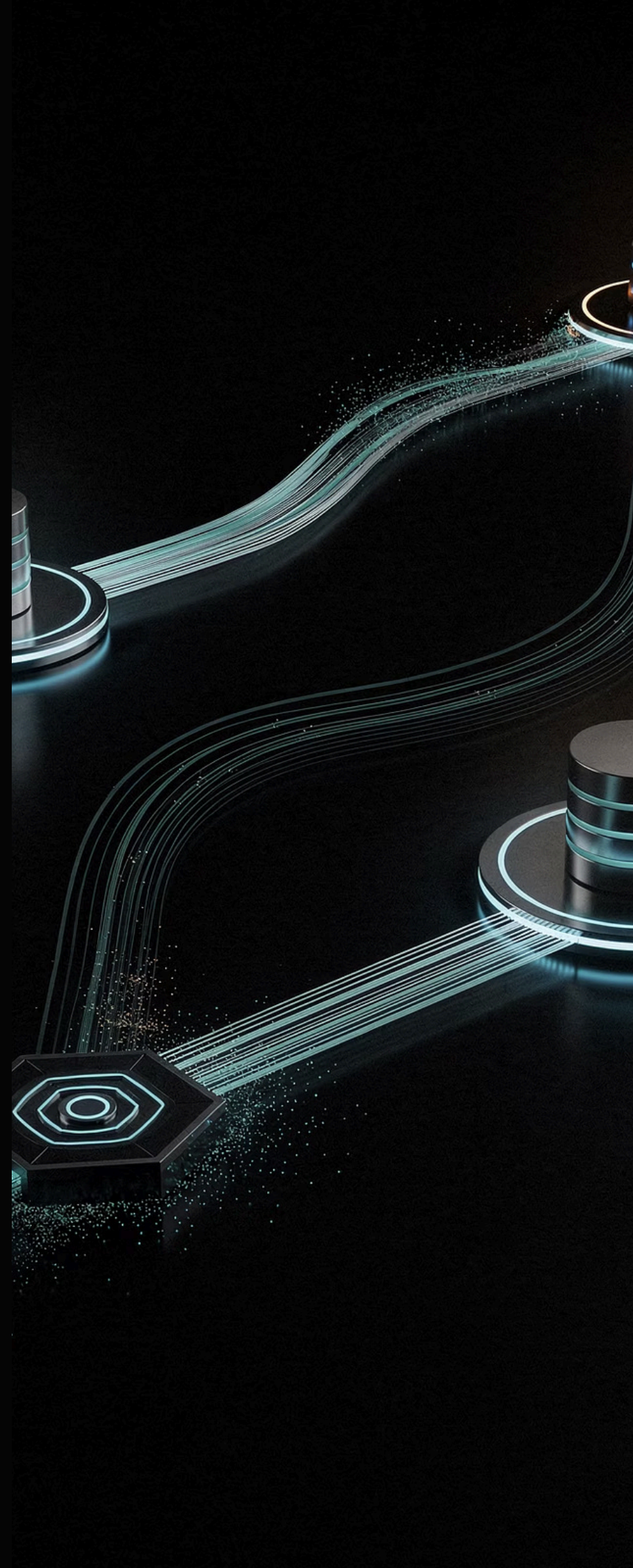
const MONGO_URI = process.env.MONGO_URI ||
'mongodb://127.0.0.1:27017/taskboard';

async function connect() {
  try {
    await mongoose.connect(MONGO_URI);
    console.log('MongoDB connected:', mongoose.connection.host);
  } catch (err) {
    console.error('MongoDB connection failed:', err.message);
    process.exit(1); // no point starting without a database
  }
}

// Graceful disconnect (used in shutdown handler)
async function disconnect() {
  await mongoose.disconnect();
  console.log('MongoDB disconnected');
}

module.exports = { connect, disconnect };
```

- ❏ Why process.exit(1) on failure? If the database is unavailable at startup, the server cannot serve meaningful requests. Failing loudly lets a process manager (PM2, Kubernetes) restart the service rather than silently serving errors.



Defining a Mongoose Schema and Model

```
// models/Task.js
const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, 'Title is required'],
      trim: true,
      minlength: [1, 'Title cannot be empty'],
      maxlength: [200, 'Title is too long'],
    },
    done: {
      type: Boolean,
      default: false,
    },
  },
  {
    timestamps: true, // adds createdAt and updatedAt automatically
  }
);

// Transform _id to id in JSON output (matches existing API contract)
taskSchema.set('toJSON', {
  transform: (_doc, ret) => {
    ret.id = ret._id;
    delete ret._id;
    delete ret.__v;
    return ret;
  },
});

const Task = mongoose.model('Task', taskSchema);

module.exports = Task;
```

- ❏ The toJSON transform is critical. Your REST API and Socket.IO clients currently receive { id: 123456, title: '...', done: false }. MongoDB uses _id. The transform converts the output so the rest of the application — routes, sockets, browser client — never needs to change.

Core Mongoose Operations

```
// READ all
const tasks = await Task.find();

// READ one
const task = await Task.findById(id);

// CREATE
const newTask = await Task.create({ title: 'Buy milk' });

// UPDATE
const updated = await Task.findByIdAndUpdate(
  id,
  { done: true },
  { new: true } // return the updated document, not the original
);

// DELETE
await Task.findByIdAndDelete(id);
```

Mongoose vs the old pattern

Old (file)

readTasks() → filter in JS

tasks.find(t => t.id === id)

Assign id: Date.now()

writeTasks(tasks)

New (Mongoose)

Task.find({ done: true })

Task.findById(id)

MongoDB auto-generates _id

await task.save()

Swapping the Storage Layer

The Architecture Decision

The service layer (`services/taskService.js`) calls two functions: `readTasks()` and `writeTasks()`. Today you replace those two functions — and nothing else.

BEFORE (Day 3-4)

`storage/fileStorage.js`

`readTasks()`

`writeTasks()`

`services/taskService.js`

`socket/taskSocket.js`

`routes/taskRoutes.js`

UNCHANGED

AFTER (Day 5)

`storage/mongoStorage.js` ← NEW

`readTasks()`

← same interface

`writeTasks()`

← same interface

`services/taskService.js` ← UNCHANGED

`socket/taskSocket.js` ← UNCHANGED

`routes/taskRoutes.js` ←

📄 Change the `require` in `taskService.js` from `./fileStorage` to `./mongoStorage`. That is the only change in the service layer.

The New Storage Module

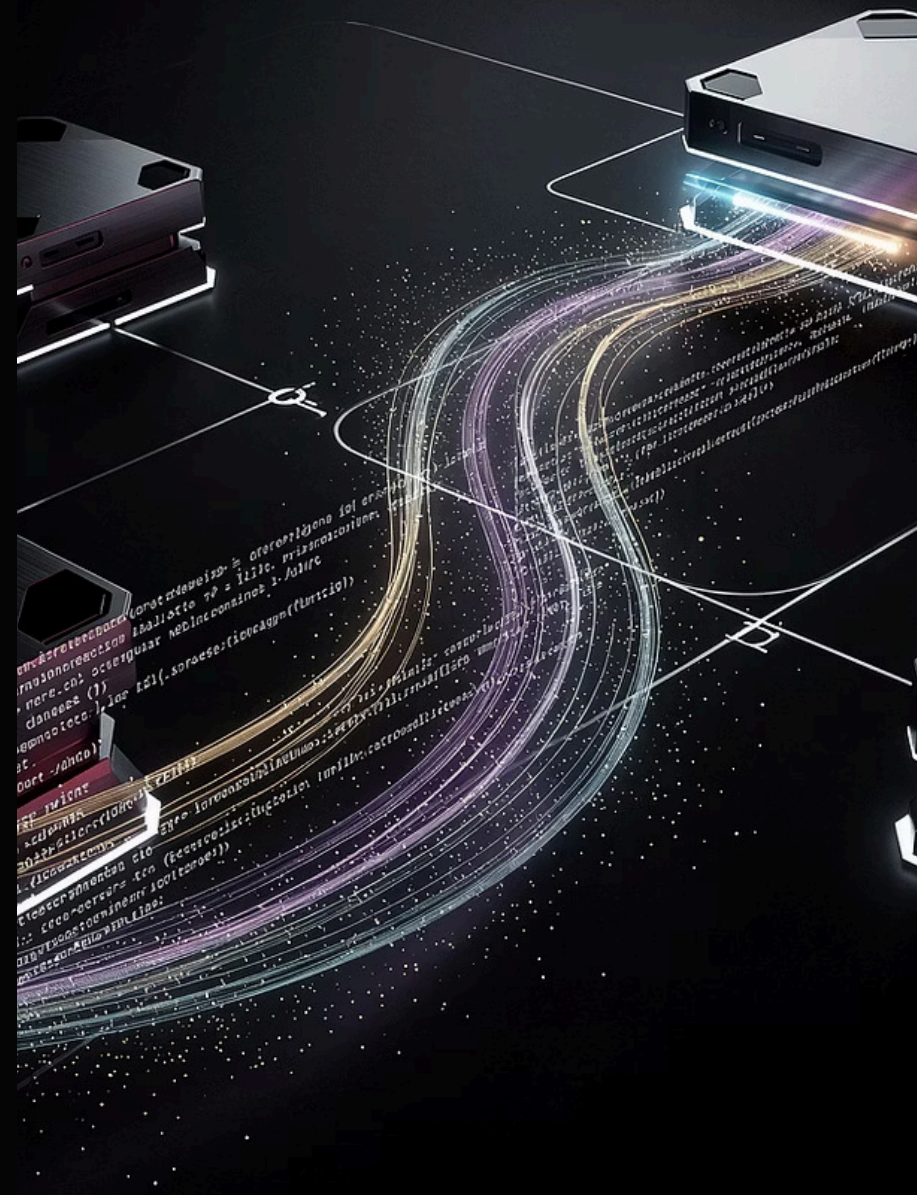
```
// storage/mongoStorage.js
const Task = require('../models/Task');

// Returns plain JS objects (matching the old fileStorage contract)
async function readTasks() {
  const tasks = await Task.find().sort({ createdAt: 1 });
  return tasks.map(t => t.toJSON());
}

// Not needed for individual writes, but kept for compatibility
// (taskService.saveTasks is called by socket toggle handler)
async function writeTasks(tasks) {
  // writeTasks with a full array is only used by the toggle handler.
  // For MongoDB we handle each document individually.
  // This is a transitional shim — Day 5 refactors the service layer properly.
  for (const taskData of tasks) {
    await Task.findByIdAndUpdate(taskData.id, {
      title: taskData.title,
      done: taskData.done,
    });
  }
}

module.exports = { readTasks, writeTasks };
```

- ❏ writeTasks is a deliberate transitional shim. The socket toggle handler passes the full array because the file-based approach had no per-document update. In the refactored service layer below, you replace the shim with proper per-document operations.



Refactoring the Service Layer for MongoDB

While readTasks and writeTasks maintain backward compatibility, MongoDB enables much cleaner per-document operations. Refactor taskService.js to use Mongoose directly:

```
// services/taskService.js (Day 5 refactor)
const Task = require('../models/Task');

async function getTasks() {
  const tasks = await Task.find().sort({ createdAt: 1 });
  return tasks.map(t => t.toJSON());
}

async function addTask(title) {
  if (!title?.trim()) throw new Error('Title is required');
  const task = await Task.create({ title: title.trim() });
  return task.toJSON();
}

async function deleteTask(id) {
  const result = await Task.findByIdAndDelete(id);
  if (!result) throw new Error('Task not found');
}

async function toggleTask(id) {
  const task = await Task.findById(id);
  if (!task) throw new Error('Task not found');
  task.done = !task.done;
  await task.save();
  return task.toJSON();
}

async function updateTask(id, { title, done }) {
  const task = await Task.findByIdAndUpdate(
    id,
    { ...(title !== undefined && { title }), ...(done !== undefined && { done }) },
    { new: true, runValidators: true }
  );
  if (!task) throw new Error('Task not found');
  return task.toJSON();
}

module.exports = { getTasks, addTask, deleteTask, toggleTask, updateTask };
```

Updated Socket Handler & Express Routes

Updated toggle handler (socket/taskSocket.js)

```
// socket/taskSocket.js — updated toggle handler
socket.on('task:toggle', async ({ id }) => {
  try {
    const updatedTask = await taskService.toggleTask(id);
    io.emit('task:updated', updatedTask);
  } catch (err) {
    socket.emit('error', { message: err.message || 'Failed
to update task' });
  }
});
```

No more loading the full array, finding by index, mutating, and writing back. One database call, atomic by default.

Updated routes (routes/taskRoutes.js)

```
router.put('/:id', async (req, res) => {
  try {
    const task = await
taskService.updateTask(req.params.id, req.body);
    res.json(task);
  } catch (err) {
    const status = err.message === 'Task not found' ? 404
: 500;
    res.status(status).json({ error: err.message });
  }
});

router.delete('/:id', async (req, res) => {
  try {
    await taskService.deleteTask(req.params.id);
    res.json({ deleted: true });
  } catch (err) {
    const status = err.message === 'Task not found' ? 404
: 500;
    res.status(status).json({ error: err.message });
  }
});
```

❏ req.params.id is now passed directly as a string —
Mongoose handles ObjectId conversion internally. No
Number() conversion.

The Updated Server Entry Point

```
// server-express.js (Day 5)
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const { connect, disconnect } = require('./storage/db');
const taskRoutes = require('./routes/taskRoutes');
const { logRequest } = require('./logger');

async function startServer() {
  // Connect to MongoDB before accepting traffic
  await connect();

  const app = express();
  const httpServer = http.createServer(app);
  const io = new Server(httpServer, { cors: { origin: '*' } });

  app.use(express.json());
  app.use(express.static('public'));
  app.use((req, _res, next) => { logRequest(req); next(); });

  app.use('/tasks', taskRoutes);

  // Health endpoint (Block 4)
  app.get('/health', (req, res) => {
    const memMB = (process.memoryUsage().heapUsed / 1024 / 1024).toFixed(1);
    res.json({
      status: 'ok',
      uptime: Math.floor(process.uptime()),
      memoryMB: Number(memMB),
      dbState: require('mongoose').connection.readyState === 1 ? 'connected' : 'disconnected',
    });
  });

  require('./socket/taskSocket')(io);

  const PORT = process.env.PORT || 3000;
  httpServer.listen(PORT, () => console.log(`Server ready on http://localhost:${PORT}`));

  // Graceful shutdown (Block 5)
  async function shutdown(signal) {
    console.log(`\n${signal} received — shutting down gracefully`);
    httpServer.close(async () => {
      await disconnect();
      process.exit(0);
    });
  }

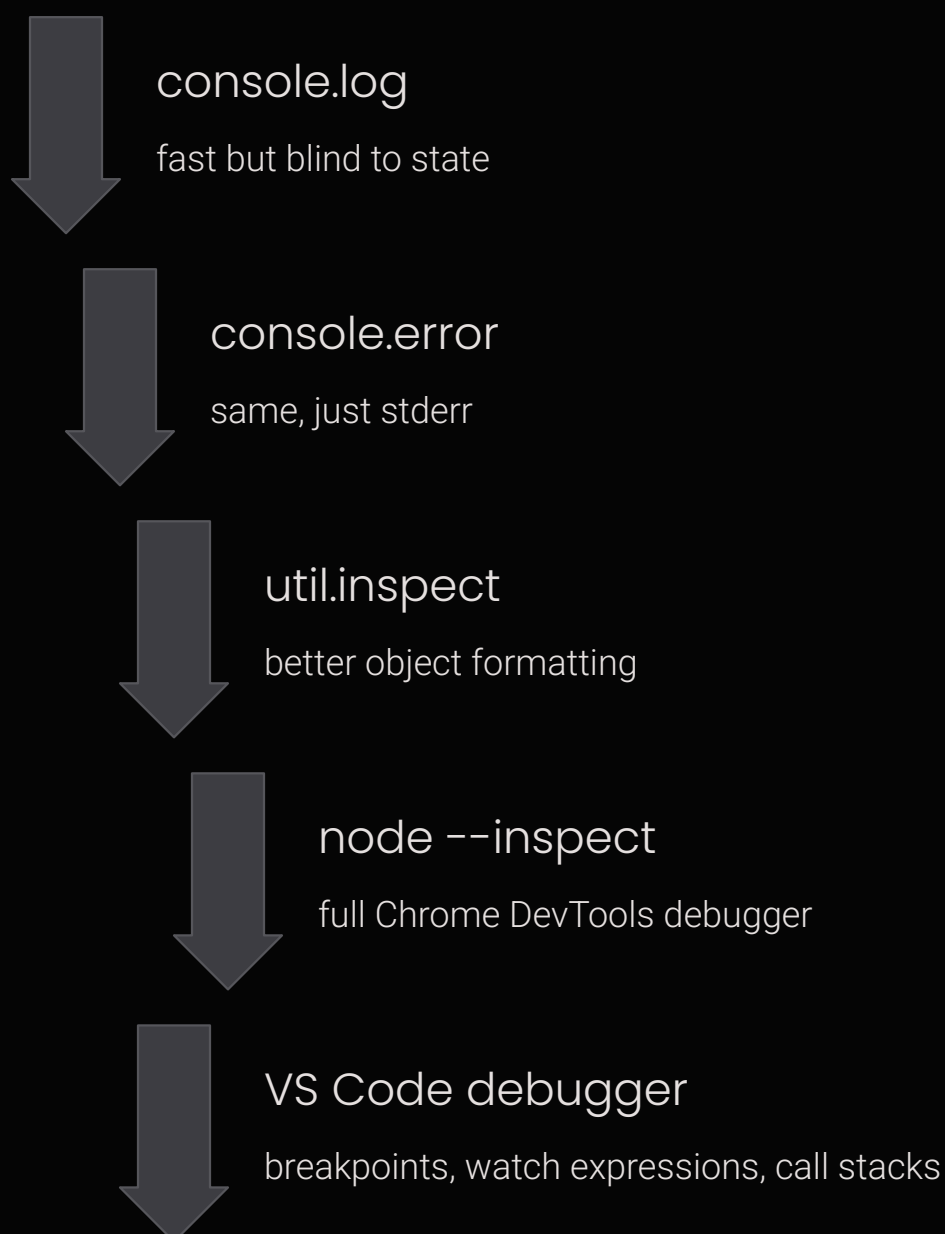
  process.on('SIGTERM', () => shutdown('SIGTERM'));
  process.on('SIGINT', () => shutdown('SIGINT'));
}

startServer().catch(err => {
  console.error('Failed to start server:', err);
  process.exit(1);
});
```


Debugging Node.js

The Debugging Ladder

Most developers rely too heavily on `console.log`. That works for simple cases but breaks down when the bug is in async code, you need to inspect a live object, or the bug only appears under load.



Running Node with the Inspector

```
# Start with inspector open (waits for a debugger to attach)
node --inspect server-express.js

# Or break immediately on first line
node --inspect-brk server-express.js
```

Then open Chrome and navigate to `chrome://inspect`. Your server appears under "Remote Target". Click Inspect.



VS Code Debugger Configuration

Create `vscode/launch.json` in your project root:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "node",
      "request": "launch",
      "name": "Debug Task Server",
      "program": "${workspaceFolder}/server-express.js",
      "env": {
        "MONGO_URI": "mongodb://127.0.0.1:27017/taskboard",
        "PORT": "3000"
      },
      "console": "integratedTerminal",
      "restart": true,
      "runtimeExecutable": "nodemon"
    },
    {
      "type": "node",
      "request": "attach",
      "name": "Attach to Running Server",
      "port": 9229
    }
  ]
}
```

Breakpoints in Practice

Set a breakpoint inside `addTask` in `taskService.js`. Then send a POST `/tasks` request via Postman. VS Code pauses execution at that line.

You can:

- Hover over any variable to inspect its value
- Watch panel — add title, task to watch expressions
- Call Stack — see exactly how you got here (route handler → service → here)
- Step Over (F10) — execute the current line and move to the next
- Step Into (F11) — follow the function call into its body
- Continue (F5) — resume until the next breakpoint

Conditional Breakpoints & Debugging Async Code

Conditional Breakpoints

Right-click a breakpoint gutter in VS Code and choose Edit Breakpoint. Add a condition:

```
title.length > 100
```

The debugger now only pauses when that condition is true — essential for bugs that only surface with specific inputs.

Debugging Async Code

The call stack in async code can look surprising. A breakpoint inside an async function shows the async chain, not the original caller. The key: look at the Async section of the call stack panel in VS Code — it shows the full chain including the event loop entry point.

Common async debugging mistakes

```
//
```

✗ This won't catch the error — the async callback is detached

```
app.get('/tasks', (req, res) => {  
  taskService.getTasks().then(tasks => res.json(tasks));  
  // if getTasks rejects, nothing catches it  
});
```

```
//
```

✓ Use async/await so try/catch works

```
app.get('/tasks', async (req, res) => {  
  try {  
    const tasks = await taskService.getTasks();  
    res.json(tasks);  
  } catch (err) {  
    res.status(500).json({ error: err.message });  
  }  
});
```


Monitoring & Health

Why Monitoring Matters

A server that crashes is obvious. A server that slowly degrades — growing memory, increasing response times, event loop lag — is invisible without monitoring. By the time users notice, the problem has been building for hours.

The /health Endpoint

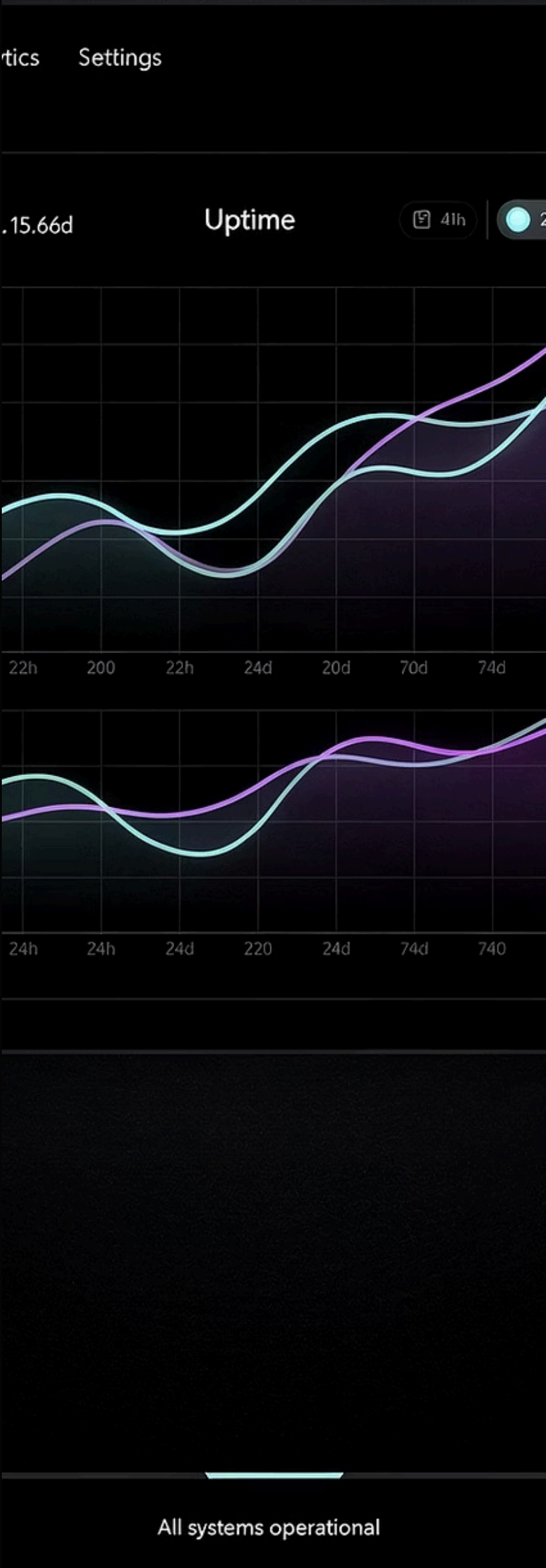
Every production service should expose a health endpoint. Orchestrators (Kubernetes, load balancers) call it to determine whether to route traffic.

```
app.get('/health', async (req, res) => {
  const mem = process.memoryUsage();
  const mongoose = require('mongoose');

  res.json({
    status: 'ok',
    uptime: Math.floor(process.uptime()), // seconds since server
    started
    memory: {
      heapUsedMB: (mem.heapUsed / 1024 / 1024).toFixed(1),
      heapTotalMB: (mem.heapTotal / 1024 / 1024).toFixed(1),
      rssMB: (mem.rss / 1024 / 1024).toFixed(1), // resident set size —
    total process memory
    },
    database: {
      state: mongoose.connection.readyState === 1 ? 'connected' :
'disconnected',
      host: mongoose.connection.host,
    },
    tasks: {
      total: await require('./models/Task').countDocuments(),
      done: await require('./models/Task').countDocuments({ done:
true }),
    },
    timestamp: new Date().toISOString(),
  });
});
```

Sample response

```
{
  "status": "ok",
  "uptime": 3847,
  "memory": { "heapUsedMB": "24.3", "heapTotalMB": "56.0", "rssMB":
"71.2" },
  "database": { "state": "connected", "host": "127.0.0.1" },
  "tasks": { "total": 18, "done": 7 },
  "timestamp": "2025-06-04T14:22:33.000Z"
}
```



Nodemon & Event Loop Lag

Nodemon for Development

Nodemon restarts the server whenever a file changes. Essential for development.

```
npm install --save-dev nodemon
```

```
{
  "scripts": {
    "start": "node server-express.js",
    "dev": "nodemon server-express.js",
    "dev:debug": "nodemon --inspect server-express.js"
  }
}
```

```
npm run dev      # development with auto-restart
npm run dev:debug # development + debugger attached
```

Event Loop Lag

The event loop processes callbacks one at a time. If a callback takes too long, subsequent callbacks wait. This is called event loop lag — visible as slow responses under load.

```
// Detect event loop lag
let lastCheck = Date.now();

setInterval(() => {
  const now = Date.now();
  const lag = now - lastCheck - 1000; // expected: 0ms
  if (lag > 100) {
    console.warn(`Event loop lag: ${lag}ms`);
  }
  lastCheck = now;
}, 1000);
```

```
//
```

❌ Synchronous JSON parsing of a large payload blocks the loop

```
const data =
JSON.parse(fs.readFileSync('huge-file.json'));
```

// ❌ Compute-heavy work on the main thread

```
function expensiveSort(arr) { return
arr.sort(/* complex compare */); }
```

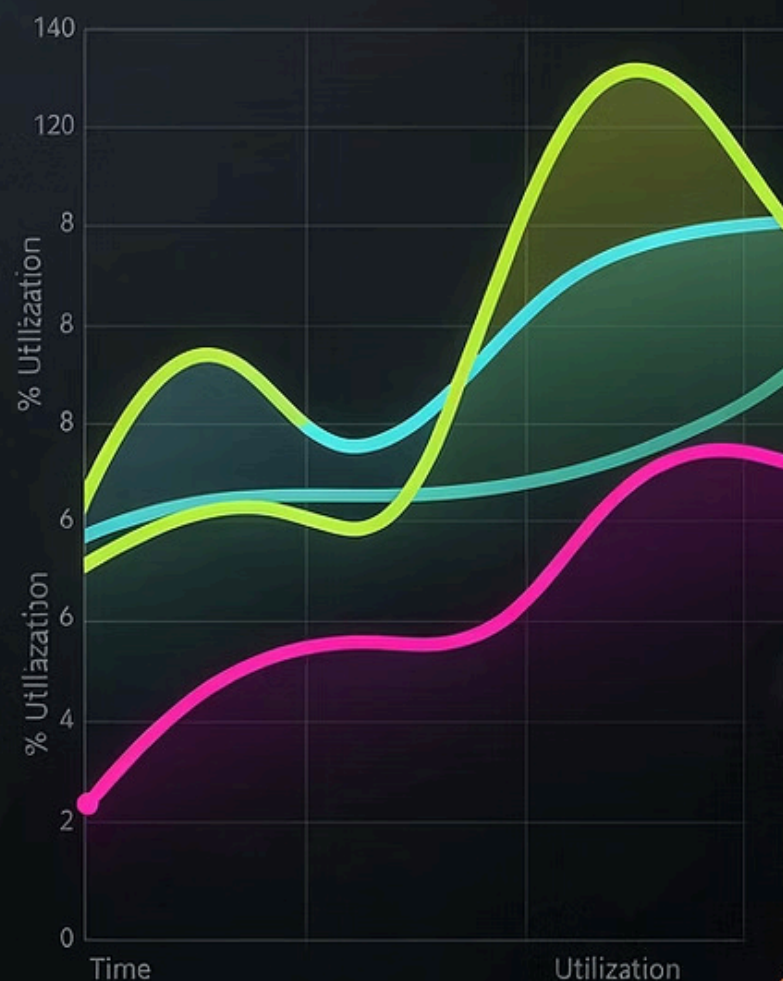
// ✅ Offload CPU work to a worker thread or child process

```
const { Worker } =
require('worker_threads');
```

Performance

Alerts

Server Load



Production Hardening

Environment Variables

Never hardcode secrets, ports, or connection strings. Use environment variables.

```
# .env (never commit this file)
MONGO_URI=mongodb://127.00.0.1:27017/taskboard
PORT=3000
NODE_ENV=development
```

```
// Load at the very top of server-express.js, before any other require
require('dotenv').config();

const MONGO_URI = process.env.MONGO_URI;
const PORT = process.env.PORT || 3000;
const NODE_ENV = process.env.NODE_ENV || 'development';

if (!MONGO_URI) {
  console.error('FATAL: MONGO_URI environment variable is not set');
  process.exit(1);
}
```

```
# .gitignore
node_modules/
.env
data/
logs/
```

npm install dotenv



Structured Error Handling

An Express error-handling middleware catches any error passed to `next(err)`. This centralises error logging and prevents error details from leaking to clients in production.

```
// errorHandler.js
function errorHandler(err, req, res, next) {
  // Mongoose validation error → 400
  if (err.name === 'ValidationError') {
    const messages = Object.values(err.errors).map(e => e.message);
    return res.status(400).json({ error: messages.join(', ') });
  }

  // Mongoose cast error (invalid ObjectId format) → 400
  if (err.name === 'CastError') {
    return res.status(400).json({ error: 'Invalid ID format' });
  }

  // MongoDB duplicate key → 409
  if (err.code === 11000) {
    return res.status(409).json({ error: 'Duplicate entry' });
  }

  const status = err.status || 500;
  const isProduction = process.env.NODE_ENV === 'production';

  // Always log the full error on the server
  console.error(`[${new Date().toISOString()}] ${req.method} ${req.url}`, err);

  // Never expose stack traces in production
  res.status(status).json({
    error: err.message || 'Internal server error',
    ...(isProduction ? {} : { stack: err.stack }),
  });
}

module.exports = errorHandler;
```

```
// server-express.js — register AFTER all routes
const errorHandler = require('./errorHandler');
app.use(errorHandler);
```

```
app.get('/tasks', async (req, res, next) => {
  try {
    const tasks = await taskService.getTasks();
    res.json(tasks);
  } catch (err) {
    next(err); // delegates to errorHandler
  }
});
```

Graceful Shutdown

When a process manager stops your server (deployment, scaling, crash recovery), it sends SIGTERM. If you just `process.exit()`, in-flight requests are dropped and database connections are not closed cleanly.

```
async function shutdown(signal) {
  console.log(`\n${signal} received. Starting graceful shutdown...`);

  // Stop accepting new connections
  httpServer.close(async () => {
    console.log('HTTP server closed.');
```



```
    // Close Socket.IO connections
    io.close(() => console.log('Socket.IO closed.));

    // Disconnect from MongoDB
    await disconnect();
    console.log('Database disconnected. Exiting.');
```



```
    process.exit(0);
  });

  // Force shutdown if graceful close takes too long
  setTimeout(() => {
    console.error('Graceful shutdown timed out. Forcing exit.');
```



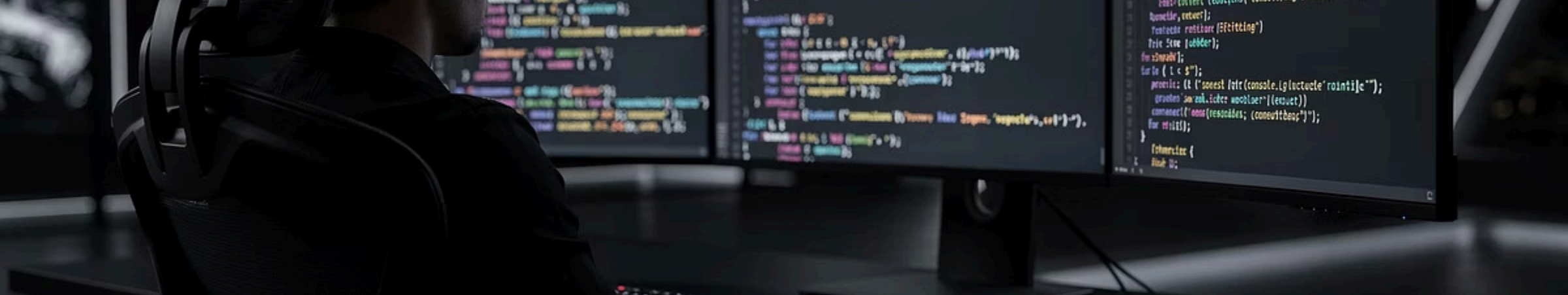
```
    process.exit(1);
  }, 10_000);
}

process.on('SIGTERM', () => shutdown('SIGTERM'));
process.on('SIGINT', () => shutdown('SIGINT')); // Ctrl+C in development
```

Final Project Structure

Code block showing the complete file tree:

```
task-board/
├── .env                # environment variables (not committed)
├── .gitignore
├── package.json
├── server-express.js   # entry point — connects DB, starts server
├── errorHandler.js    # centralised error middleware
├── logger.js          # stream-based request logging
├── models/
│   └── Task.js        # Mongoose schema + model
├── storage/
│   └── db.js          # MongoDB connect / disconnect
├── services/
│   └── taskService.js  # business logic — getTasks, addTask, etc.
├── routes/
│   └── taskRoutes.js   # REST endpoints
├── socket/
│   └── taskSocket.js   # Socket.IO event handlers
├── public/
│   └── index.html      # browser client
└── logs/
    └── server.log      # written by logger.js
```

HANDS-ON LABS

Hands-On Labs

Four progressive labs that take you from a bare MongoDB connection all the way to a fully production-hardened task board.

1

Lab 1 — Install MongoDB & Connect

Get MongoDB running locally and connect the app. Verify the log: MongoDB connected: 127.0.0.1

Duration: ~20 min

2

Lab 2 — Swap the Storage Layer

Replace fileStorage.js with mongoStorage.js and verify the full API still works. Test all CRUD endpoints with Postman.

Duration: ~25 min

3

Lab 3 — Debug a Live Request

Use VS Code breakpoints to inspect a task being created. Set a conditional breakpoint on `title.length > 10`.

Duration: ~20 min

4

Lab 4 — Add the Health Endpoint & Nodemon

Add GET `/health` and configure Nodemon. Verify auto-restart and read the health response.

Duration: ~15 min

📌 Optional Challenge: Rate Limiting — add basic rate limiting to the REST API without installing a library.

Using URI: mondobb+srv//app-user@prod-cluster-01/

Authentication: SCRAM-SHA-256

Status: connection established

Repnary: db-node-01:27017

Database: analytics

Ping: 2.1 ms

LAB 1 · ~20 MIN

Install MongoDB & Connect

Goal: Get MongoDB running locally and connect the app.

Install MongoDB Community Edition (or use MongoDB Atlas free tier)

Start the local MongoDB service

Install dependencies: npm install mongoose dotenv

Create .env with MONGO_URI, PORT, NODE_ENV

Create storage/db.js and models/Task.js as shown in Block 1

In server-express.js, call await connect() before httpServer.listen()

Start the server and verify the log

```
# macOS (Homebrew)
brew services start mongodb-community

# Linux
sudo systemctl start mongod

# Verify it's running
mongosh --eval "db.adminCommand('ping')"
```



MongoDB connected: 127.0.0.1
Server ready on http://localhost:3000



LAB 2 · ~25 MIN

Swap the Storage Layer

Goal: Replace fileStorage.js with mongoStorage.js and verify the full API still works.

Create services/taskService.js

Using Mongoose directly (Block 2)

Update routes/taskRoutes.js

Remove Number() cast on IDs

Update socket/taskSocket.js

Replace toggle handler to use taskService.toggleTask

Test all endpoints with Postman

(curl commands below)

Open two browser tabs

Create a task in one – verify it appears in both (Socket.IO still works)

Restart the server

Tasks persist (they're in MongoDB, not memory)

Create a task

```
curl -X POST http://localhost:3000/tasks \
-H "Content-Type: application/json" \
-d '{"title": "Buy milk"}'
```

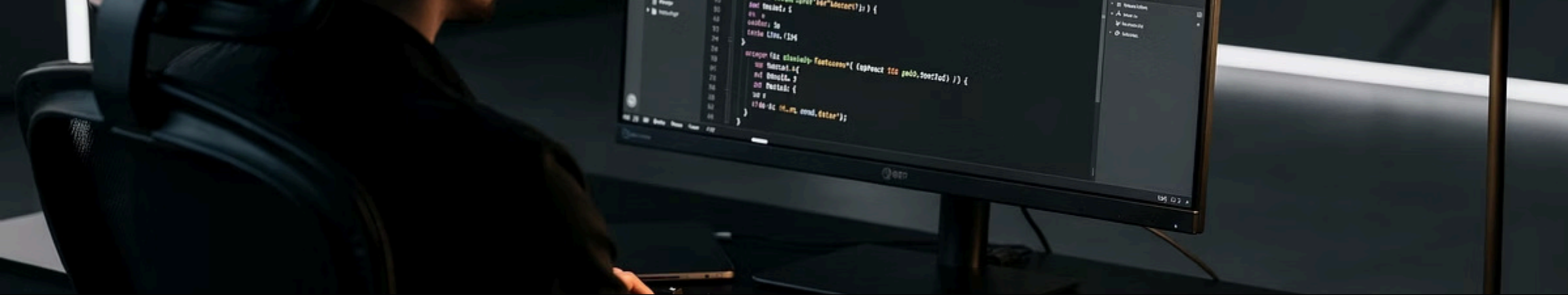
Get all tasks

```
curl http://localhost:3000/tasks
```

Delete using the MongoDB _id returned from create

```
curl -X DELETE http://localhost:3000/tasks/<id>
```

Expected: All CRUD operations work. Data survives restarts.



LAB 3 · ~20 MIN

Debug a Live Request

Goal: Use VS Code breakpoints to inspect a task being created.

Create `.vscode/launch.json` as shown in Block 3

Start the server using VS Code's Run & Debug panel (not the terminal)

Set a breakpoint on the `Task.create(...)` line inside `addTask` in `taskService.js`

Send a POST `/tasks` request from Postman

VS Code pauses at your breakpoint. Inspect: `title` in the Variables panel, add `title` and `task` to the Watch panel, look at the Call Stack — identify the route handler that called `addTask`

Press F10 to step over the `create` call and watch `task` populate

Press F5 to resume and verify Postman received a 201 response

📌 Bonus: Set a conditional breakpoint on `title.length > 10`. Send a short title — execution should not pause. Send a long one — it should.



LAB 4 · ~15 MIN

Add the Health Endpoint & Nodemon

Goal: Add GET /health and configure Nodemon.

Add the /health route to server-express.js (Block 4)

Install Nodemon: npm install --save-dev nodemon

Update package.json scripts with start, dev, and dev:debug commands

Run with npm run dev

Edit any file — verify the server restarts automatically

Call GET http://localhost:3000/health in Postman and read the response

```
{
  "scripts": {
    "start": "node server-express.js",
    "dev": "nodemon server-express.js",
    "dev:debug": "nodemon --inspect server-express.js"
  }
}
```

Expected response includes: uptime, memory, DB connection state, task counts.

Optional Challenge — Rate Limiting

```
// middleware/rateLimit.js
const requests = new Map(); // ip → { count, resetAt }

function rateLimit(maxRequests = 100, windowMs = 60_000) {
  return (req, res, next) => {
    const ip = req.ip;
    const now = Date.now();
    const entry = requests.get(ip);

    if (!entry || now > entry.resetAt) {
      requests.set(ip, { count: 1, resetAt: now + windowMs });
      return next();
    }

    if (entry.count >= maxRequests) {
      return res.status(429).json({
        error: 'Too many requests',
        retryAfter: Math.ceil((entry.resetAt - now) / 1000),
      });
    }

    entry.count++;
    next();
  };
}

module.exports = rateLimit;
```

```
// server-express.js
const rateLimit = require('./middleware/rateLimit');
app.use('/tasks', rateLimit(50, 60_000)); // 50 requests per minute
```



INSTRUCTOR NOTES

Common Student Mistakes

1. Using `Number(req.params.id)` with MongoDB ObjectIds

MongoDB IDs are 24-character hex strings, not numbers. Calling `Number()` returns `NaN`, and `Task.findById(NaN)` returns `null`. The fix: pass `req.params.id` directly as a string. Mongoose casts it to `ObjectId` internally — and throws a `CastError` if the format is invalid, which your error handler should catch.

2. Calling `app.listen` instead of `httpServer.listen` (still happens on Day 5)

Remind students this was the critical `Socket.IO` mistake from Day 4. Nothing changes on Day 5 — the pattern stays the same.

3. Not awaiting `connect()` before `httpServer.listen()`

Without `await connect()`, the server starts accepting requests before MongoDB is ready. The first few POST requests throw connection errors. The fix is wrapping startup in an `async` function and awaiting the database connection first.

4. Forgetting `.toJSON()` on Mongoose documents

Mongoose documents are not plain JS objects. The `toJSON` transform on the schema (Block 1) handles this globally — but students sometimes bypass it by returning the raw document without `.toJSON()`.

5. Mutating tasks in the socket handler instead of using `toggleTask`

Some students keep the old toggle pattern (load all, find, mutate, saveTasks). That now calls the shim `writeTasks`, which iterates and calls `findByIdAndUpdate` on every task — a DB call per task on every toggle. The correct pattern: one `findById` + `save()` via `toggleTask`.

DAY 5 OUTCOMES

What You've Built Today

Students leave with a production-shaped application and the habits to maintain it.



MongoDB + Mongoose integration

Real persistence that survives restarts and scales



Surgical storage layer swap

Proof that the architecture from Days 1–4 was correct



VS Code debugger workflow

Replaces console.log as the primary debugging tool



/health endpoint

Foundation for any monitoring or alerting setup



Graceful shutdown

Prevents data loss during deployments



Environment variables + .env

Production-safe configuration management



Centralised error handler

One place to log, classify, and respond to errors

"The measure of good architecture is not how well it handles what you planned for. It's how easily it handles what you didn't."

Final Outcome of the Week

Students have built a Real-Time Task Board that includes:

Day 1 — Node.js CLI

Day 2 — REST API with the http module

Day 3 — Modular Express architecture
with async patterns

Day 4 — Real-time collaboration via
Socket.IO

Day 5 — MongoDB persistence with
Mongoose + Debugging, monitoring, and
production practices

Every day's work built directly on the previous day. The architecture introduced on Day 3 proved its value on Day 5: swapping the entire persistence layer required touching exactly one file in the service layer and one module in storage. Everything else — routes, sockets, browser client — continued working without modification.

"The measure of good architecture is not how well it handles what you planned for. It's how easily it handles what you didn't."